Midterm Learning Report

Data Structures and Algorithms

This report reflects how my approach to programming problems has developed during the course. At the beginning, I mainly focused on obtaining the correct output and often considered a problem complete once it passed the sample test cases. While attempting more questions, I began encountering Time Limit Exceeded errors, invalid indexing, repeated calculations, unnecessary memory usage, and recursive functions that did not stop correctly. These experiences made me realise that a correct answer is only one part of a complete solution.
I am now reasonably comfortable with several easy-level questions, but medium-level problems are still challenging when the required pattern is not immediately visible. In such situations, I have sometimes used ChatGPT to understand the basic intuition, identify a suitable technique, or see how a larger problem can be divided into smaller parts. I then try to trace the algorithm manually, rewrite it in my own style, and understand its complexity and correctness rather than directly copying a finished answer. This has helped me build a better foundation, although recognising medium-level patterns independently is still something I am working on.
I have also recently completed three introductory Dynamic Programming problems: Climbing Stairs, Min Cost Climbing Stairs, and House Robber. These problems introduced me to states, recurrence relations, base cases, overlapping subproblems, and the idea of storing previous results instead of calculating them repeatedly. Advanced Dynamic Programming has not yet been covered, but the basic problems helped me connect recursion with optimisation.
Rather than only listing the topics covered, this report discusses the ideas, mistakes, mathematical reasoning, correctness proofs, and practical observations that contributed most to my learning.

Name: Aditya Murukate

Roll Number: 24B2245

Course: Data Structures and Algorithms

Submission: Midterm Learning Report

June 24, 2026

Contents

1. Introduction and Development of My Approach

Before this course, I often treated programming as a direct translation from an idea to code. I would read a question, think of the first approach that seemed reasonable, and immediately begin implementing it. This worked for simple traversal questions, but it became unreliable as the questions became longer or involved more than one condition.

In medium-level problems, the main difficulty was usually not Python syntax. The harder part was recognising the structure of the question. It was not always clear whether a problem required a sliding window, a prefix sum, binary search, recursion, a stack, a heap, or a combination of techniques.

I gradually started following a more structured process:

1. Read the problem carefully and identify the exact input and output.
2. Examine the constraints before selecting an approach.
3. Think of the simplest correct method first.
4. Calculate its approximate time and space complexity.
5. Identify repeated calculations or repeated searches.
6. Consider whether previous information can be stored.
7. Select a suitable data structure or algorithm.
8. Test the solution on ordinary and boundary cases.
9. Explain why the algorithm is correct.

This process has made unfamiliar questions less overwhelming. Even when I cannot immediately solve a medium-level problem, I can usually divide it into smaller questions and understand which part is causing difficulty.

1.1 Understanding an Explanation Versus Producing a Solution

One important realisation was that understanding an explanation is not the same as producing the idea independently.

After reading a sliding-window or monotonic-stack solution, the algorithm may look straightforward. However, starting from only the problem statement and discovering that pattern is much harder.

I therefore try to separate two stages of learning:

- understanding why an existing solution works;
- learning to recognise when that technique should be applied.

When I use ChatGPT, I try to use it mainly for the first stage. I may ask for the base intuition, the repeated work in my current solution, or the meaning of a state or invariant. After that, I try to rewrite and trace the algorithm myself.

1.2 Correctness Is Not the Final Step

A program can produce correct answers for the examples and still fail during submission. A complete solution must consider

$$\text{correctness}, \quad \text{time complexity}, \quad \text{space complexity}, \quad \text{edge cases}.$$

Common reasons for failure include:

- the algorithm is too slow for the maximum input size;
- the program creates too many temporary lists;
- recursion becomes too deep;

- an index moves outside the valid range;
- an empty stack or queue is accessed;
- shared data is modified unintentionally;
- an empty or one-element input is not handled.

I now consider complexity analysis and testing as part of the solution rather than something added after the program is finished.

2. Multiple Solutions and Complexity Analysis

One of the most useful lessons from the course is that a single problem can have several correct solutions. These solutions may differ in running time, memory usage, clarity, and ease of implementation.

2.1 Example: Contains Duplicate

Suppose the task is to determine whether a list contains a repeated value.

The most direct method compares every possible pair.

```
def contains_duplicate(nums):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] == nums[j]:
                return True

    return False
```

For the first value, approximately $n-1$ comparisons are performed. For the second value, $n-2$ comparisons are performed, and so on. Therefore, the total number of comparisons is

$$(n-1)+(n-2)+\cdots+2+1.$$

Using

$$1+2+\cdots+(n-1)=\frac{n(n-1)}{2},$$

we obtain

$$T(n)=O(n^2).$$

The method is correct because every possible pair of positions is checked. However, for a large input, the number of comparisons becomes too high.

2.2 Sorting-Based Solution

A second method sorts the list and compares adjacent values.

```
def contains_duplicate(nums):
    sorted_nums = sorted(nums)

    for i in range(1, len(sorted_nums)):
        if sorted_nums[i] == sorted_nums[i - 1]:
            return True

    return False
```

After sorting, equal values must appear next to each other. Therefore, checking adjacent positions is sufficient.

The sorting step takes approximately

$$O(n\log n)$$

time, and the traversal takes $O(n)$. Thus,

$$O(n\log n)+O(n)=O(n\log n).$$

2.3 Set-Based Solution

A third method uses a Python set.

```python
def contains_duplicate(nums):
    seen_numbers = set()

    for number in nums:
        if number in seen_numbers:
            return True

        seen_numbers.add(number)

    return False
```

The average cost of a set membership check and insertion is $O(1)$. Since each value is processed once,

$$T(n) = O(n).$$

The set may store up to n values, so

$$S(n) = O(n).$$

Claim. *The set-based algorithm returns* `True` *if and only if a duplicate exists.*

Proof. Suppose the algorithm returns `True` for a value x. This happens only when x is already in the set. Since the set contains only earlier values, x must have appeared before.

Conversely, suppose a duplicate value x exists. Its first occurrence is inserted into the set. When a later occurrence is processed, the membership test finds x, so the algorithm returns `True`. □

2.4 Time–Memory Trade-Off

This example shows a common trade-off:

$$\text{less execution time} \iff \text{additional memory.}$$

The set based solution is faster on average but stores previous values. The sorting solution requires sorting and creates a new list. The brute-force solution uses little extra memory but takes much longer.

2.5 Common Complexity Classes

Complexity	Meaning
$O(1)$	Constant amount of work
$O(\log n)$	Search space is repeatedly divided
$O(n)$	Every element is processed a fixed number of times
$O(n \log n)$	Efficient sorting or divide-and-conquer
$O(n^2)$	Every pair may be examined
$O(2^n)$	Every binary choice may be explored
$O(n!)$	Every possible ordering may be generated

The input constraints often suggest which complexity is expected:

- $n \leq 20$: exponential backtracking may be possible;
- $n \approx 10^3$: some $O(n^2)$ methods may work;
- $n \approx 10^5$: usually $O(n)$ or $O(n \log n)$;
- very large search space: often $O(\log n)$ or mathematical reasoning.

3. Array Traversal, Frequency Counting, and Prefix Techniques

Arrays were the starting point for many of the problems. The main challenge was deciding what information should be maintained during traversal.

3.1 Maximum Consecutive Ones

```python
def find_max_consecutive_ones(nums):
    current_count = 0
    maximum_count = 0

    for number in nums:
        if number == 1:
            current_count += 1
        else:
            current_count = 0

        maximum_count = max(
            maximum_count,
            current_count
        )

    return maximum_count
```

Invariant. *After processing a position,* `current_count` *stores the number of consecutive ones ending exactly at that position.*

If the current value is 1, it extends the previous sequence. If the value is 0, no sequence of ones can end there, so the count becomes zero.

The maximum value observed is therefore the longest consecutive sequence.

$$T(n) = O(n), \qquad S(n) = O(1).$$

3.2 Majority Element and Cancellation

The Boyer–Moore voting algorithm uses a cancellation argument.

```python
def majority_element(nums):
    candidate = None
    count = 0

    for number in nums:
        if count == 0:
            candidate = number

        if number == candidate:
            count += 1
        else:
            count -= 1

    return candidate
```

Suppose the majority element appears more than $n/2$ times. Each different value can cancel one occurrence of the majority element. Since the majority appears more frequently than all other values combined, at least one majority occurrence remains uncancelled.

The algorithm simulates this cancellation process.

$$T(n) = O(n), \qquad S(n) = O(1).$$

3.3 Kadane's Algorithm

Kadane's algorithm finds the maximum sum of a contiguous subarray.

```python
def maximum_subarray(nums):
    current_sum = nums[0]
    maximum_sum = nums[0]

    for i in range(1, len(nums)):
        current_sum = max(
            nums[i],
            current_sum + nums[i]
        )

        maximum_sum = max(
            maximum_sum,
            current_sum
        )

    return maximum_sum
```

The recurrence is

$$\text{current\_sum} = \max\left(\text{nums}[i], \text{current\_sum} + \text{nums}[i]\right).$$

A maximum-sum subarray ending at index i either starts at i or extends the best subarray ending at $i-1$.

Claim. *After processing index i, `current_sum` is the maximum sum of any subarray ending at i.*

Proof. Any contiguous subarray ending at i either contains index $i-1$ or does not. If it does not, its sum is $\text{nums}[i]$. If it does, the best previous part is the maximum-sum subarray ending at $i-1$.

The recurrence selects the better possibility. □

3.4 Product of Array Except Self

My original approach repeatedly calculated the product of values before and after each position. It was correct but repeated many calculations.

The improved observation is

$$\text{answer}[i] = \left(\prod_{j=0}^{i-1} \text{nums}[j]\right)\left(\prod_{j=i+1}^{n-1} \text{nums}[j]\right).$$

```python
def product_except_self(nums):
    n = len(nums)

    left_products = [1] * n
    right_products = [1] * n
    answer = []

    for i in range(1, n):
        left_products[i] = (
            left_products[i - 1] * nums[i - 1]
        )

    for i in range(n - 2, -1, -1):
        right_products[i] = (
            right_products[i + 1] * nums[i + 1]
        )

    for i in range(n):
```

```
        answer.append(
            left_products[i] * right_products[i]
        )

    return answer
```

Claim. *For every index i, `left_products[i]` is the product of all values strictly before i.*

Proof. For $i = 0$, there are no earlier values, so the product is 1.
Assume the result is correct for $i - 1$. Then

$$\texttt{left\_products[i]} = \texttt{left\_products[i-1]} \cdot \text{nums}[i-1].$$

Therefore, the product now includes exactly the values from index 0 to $i - 1$. The result follows by induction. □

The same argument applies to the right products.

$$T(n) = O(n), \qquad S(n) = O(n).$$

3.5 Prefix Sums

For an array a, define

$$P[i] = a[0] + a[1] + \cdots + a[i].$$

Then the sum from index l to r is

$$P[r] - P[l-1]$$

for $l > 0$.

Proof. By definition,

$$P[r] = a[0] + \cdots + a[l-1] + a[l] + \cdots + a[r],$$

and

$$P[l-1] = a[0] + \cdots + a[l-1].$$

Subtracting cancels all terms before l, leaving

$$a[l] + \cdots + a[r].$$

□

For q range queries, prefix sums reduce the total time from approximately

$$O(nq)$$

to

$$O(n + q).$$

3.6 Prefix Remainders

If two prefix sums satisfy

$$P[j] \bmod k = P[i] \bmod k,$$

then

$$(P[j] - P[i]) \bmod k = 0.$$

Therefore, the subarray between the two positions has a sum divisible by k.
This turns an $O(n^2)$ subarray-checking problem into a frequency-counting problem.

4. Sliding Windows and Two-Pointer Methods

Sliding-window problems were among the first medium-level questions that I found difficult. The code may look short, but identifying the correct window condition is not always easy.

4.1 General Sliding-Window Structure

```
left = 0

for right in range(len(nums)):

    # Add nums[right] to the current window

    while window_is_invalid:

        # Remove nums[left] from the window
        left += 1

    # Update the answer
```

Invariant. *After the shrinking loop finishes, the current window satisfies the required condition.*

4.2 Why the Complexity Is Linear

The right pointer moves forward n times. The left pointer also moves only forward and increases at most n times.

Therefore, the total number of pointer movements is at most

$$2n.$$

Thus,

$$T(n) = O(n).$$

This is an example of amortised analysis. One iteration may move the left pointer several times, but each element enters and leaves the window at most once.

4.3 Longest Substring Without Repeated Characters

```
def length_of_longest_substring(text):
    character_count = {}
    left = 0
    longest_length = 0

    for right in range(len(text)):
        current_character = text[right]

        character_count[current_character] = (
            character_count.get(current_character, 0) + 1
        )

        while character_count[current_character] > 1:
            left_character = text[left]
            character_count[left_character] -= 1
            left += 1

        current_length = right - left + 1
        longest_length = max(
            longest_length,
            current_length
```

```
        )

    return longest_length
```

Claim. *For each right endpoint, the algorithm finds the longest valid substring ending at that endpoint.*

Proof. When a duplicate is introduced, the left pointer moves until that duplicate is removed. It stops at the first position where the window becomes valid.

Any earlier left position would still contain a repeated character. Therefore, the current left position gives the longest valid substring ending at the current right position. □

4.4 Fixed-Size Sliding Windows

For a fixed-size window of length k, the sum can be updated using

$$\text{new sum} = \text{old sum} - \text{outgoing value} + \text{incoming value}.$$

A direct method that recalculates every window sum takes

$$O(nk)$$

time. Maintaining the sum reduces the complexity to

$$O(n).$$

4.5 Two Pointers on Sorted Data

Suppose

$$a[l] + a[r] < \text{target}.$$

Moving the right pointer left would reduce the sum further. Therefore, the left pointer must move right.

Similarly, if

$$a[l] + a[r] > \text{target},$$

the right pointer must move left.

The sorted order justifies discarding one set of possibilities after each comparison.

4.6 Three Sum

The 3 Sum problem combines sorting and two pointers.

After sorting, one index is fixed, and the remaining two values are searched using left and right pointers. Sorting requires

$$O(n \log n).$$

The two-pointer search is $O(n)$ for every fixed index, giving

$$O(n^2)$$

overall. This is better than checking every triple, which takes

$$O(n^3).$$

5. Sorting Algorithms and Binary Search

Sorting algorithms were useful because they showed how different methods can produce the same output while having different performance.

5.1 Bubble Sort

```python
def bubble_sort(nums):
    n = len(nums)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if nums[j] > nums[j + 1]:
                nums[j], nums[j + 1] = (
                    nums[j + 1],
                    nums[j]
                )
                swapped = True

        if not swapped:
            break

    return nums
```

Invariant. *After k complete passes, the final k positions contain the k largest values in sorted order.*

The worst-case number of comparisons is

$$(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

Therefore,

$$T(n) = O(n^2).$$

5.2 Selection Sort

Selection sort repeatedly finds the minimum value in the unsorted portion.

Invariant. *Before iteration i, the first i positions contain the i smallest values in sorted order.*

It also takes $O(n^2)$ time but generally performs fewer swaps than bubble sort.

5.3 Insertion Sort

```python
def insertion_sort(nums):
    for i in range(1, len(nums)):
        current_value = nums[i]
        j = i - 1

        while j >= 0 and nums[j] > current_value:
            nums[j + 1] = nums[j]
            j -= 1

        nums[j + 1] = current_value

    return nums
```

Invariant. *At the beginning of iteration i, the portion from index 0 to $i-1$ is sorted.*

The worst-case complexity is

$$O(n^2),$$

but the algorithm performs well for nearly sorted data.

5.4 Merge Sort

```python
def merge_sort(nums):
    if len(nums) <= 1:
        return nums

    middle = len(nums) // 2

    left_half = merge_sort(nums[:middle])
    right_half = merge_sort(nums[middle:])

    return merge(left_half, right_half)


def merge(left_half, right_half):
    merged = []

    left_index = 0
    right_index = 0

    while (
        left_index < len(left_half)
        and right_index < len(right_half)
    ):
        if (
            left_half[left_index]
            <= right_half[right_index]
        ):
            merged.append(left_half[left_index])
            left_index += 1
        else:
            merged.append(right_half[right_index])
            right_index += 1

    merged.extend(left_half[left_index:])
    merged.extend(right_half[right_index:])

    return merged
```

The recurrence is

$$T(n) = 2T(n/2) + O(n).$$

There are approximately $\log_2 n$ levels, and each level processes n elements. Therefore,

$$T(n) = O(n \log n).$$

Claim. *Merging two sorted lists produces one sorted combined list.*

Proof. At each step, the algorithm compares the smallest unmerged value from each list. Since both lists are sorted, the smaller current value must be the smallest remaining value overall.

Selecting it preserves sorted order. Repeating the process produces a fully sorted result. □

5.5 Binary Search

```python
def binary_search(nums, target):
    left = 0
    right = len(nums) - 1

    while left <= right:
        middle = left + (right - left) // 2

        if nums[middle] == target:
            return middle

        if nums[middle] < target:
            left = middle + 1
        else:
            right = middle - 1

    return -1
```

Invariant. *If the target exists, it remains inside the current search interval.*

After k steps, the search-space size is approximately

$$\frac{n}{2^k}.$$

Setting this equal to 1 gives

$$k = \log_2 n.$$

Thus,

$$T(n) = O(\log n).$$

5.6 Binary Search on the Answer

This technique searches a numerical answer range instead of stored indices.
The required condition must be monotonic, such as

$$\text{False, False, False, True, True, True.}$$

For example, if a shipping capacity x is sufficient, every capacity larger than x is also sufficient.
The difficult part is usually defining the feasibility function and proving that the condition is monotonic.

6. Stacks, Queues, Deques, and Monotonic Structures

Stacks and queues use different ordering rules and are therefore suitable for different types of problems.

6.1 Valid Parentheses

A stack follows Last In, First Out order.

```
def is_valid_parentheses(text):
    stack = []

    matching = {
        ')': '(',
        ']': '[',
        '}': '{'
    }

    for character in text:
        if character in '([{':
            stack.append(character)
        else:
            if len(stack) == 0:
                return False

            top_bracket = stack.pop()

            if top_bracket != matching[character]:
                return False

    return len(stack) == 0
```

Claim. *The algorithm accepts exactly the correctly matched and nested bracket strings.*

Proof. Every opening bracket is pushed onto the stack. A closing bracket must match the most recently opened unmatched bracket, which is stored at the top.

If the stack is empty, no opening bracket exists. If the types do not match, the nesting is invalid.

At the end, the stack must be empty; otherwise, some opening brackets remain unclosed. □

6.2 Queues

A queue follows First In, First Out order.

Python's `collections.deque` supports efficient insertion at the back and removal from the front.

Queues are useful for:

- level-order tree traversal;
- breadth-first search;
- recent-request tracking;
- processing elements in discovery order.

6.3 Monotonic Stack

The Daily Temperatures problem stores unresolved indices in a decreasing stack.

```
def daily_temperatures(temperatures):
    answer = [0] * len(temperatures)
    stack = []

    for current_index in range(len(temperatures)):
```

```
        while (
            stack
            and temperatures[current_index]
            > temperatures[stack[-1]]
        ):
            previous_index = stack.pop()

            answer[previous_index] = (
                current_index - previous_index
            )

        stack.append(current_index)

    return answer
```

Each index is pushed once and popped at most once. Therefore, the total number of stack operations is at most $2n$.

$$T(n) = O(n).$$

6.4 Monotonic Deque

In the Sliding Window Maximum problem, a deque stores indices in decreasing order of their values.

```
from collections import deque

def max_sliding_window(nums, window_size):
    result = []
    indices = deque()

    for right in range(len(nums)):

        while (
            indices
            and indices[0] <= right - window_size
        ):
            indices.popleft()

        while (
            indices
            and nums[indices[-1]] <= nums[right]
        ):
            indices.pop()

        indices.append(right)

        if right >= window_size - 1:
            result.append(nums[indices[0]])

    return result
```

Smaller values at the back are removed because a newer and larger value will remain useful for at least as long as they do.

Each index enters and leaves the deque at most once, so

$$T(n) = O(n).$$

7. Recursion and Backtracking

Recursion became easier when I started viewing each function call as solving a smaller version of the same problem.

A valid recursive solution requires:

1. a base case;
2. progress towards the base case;
3. a correct relation between the current problem and the smaller one.

7.1 Factorial and Induction

```
def factorial(number):
    if number == 0:
        return 1

    return number * factorial(number - 1)
```

The definition is

$$n! = n(n-1)!$$

with

$$0! = 1.$$

Claim. *The recursive function returns $n!$ for every non-negative integer n.*

Proof. For $n = 0$, the function returns $1 = 0!$.

Assume that the function correctly returns $k!$. For $k + 1$, it returns

$$(k+1) \times \text{factorial}(k).$$

Using the induction assumption,

$$(k+1) \times k! = (k+1)!.$$

Therefore, the function is correct for every non-negative integer. □

7.2 Recursion Depth

Every recursive call occupies stack memory. If the recursion depth is n, the auxiliary space is generally

$$O(n).$$

Python also has a recursion-depth limit. Therefore, a logically correct recursive method may still raise a `RecursionError`.

7.3 Backtracking

Backtracking makes a choice, explores it, and then reverses it.

```
def solve(...):
    if solution_is_complete:
        save_solution()
        return

```

```
    for choice in possible_choices:
        make_choice(choice)

        solve(...)

        undo_choice(choice)
```

The undo step is essential because each branch should begin from the state that existed before the choice.

7.4 Generating Subsets

```
def subsets(nums):
    result = []
    current_subset = []

    def generate(index):
        if index == len(nums):
            result.append(current_subset[:])
            return

        current_subset.append(nums[index])
        generate(index + 1)
        current_subset.pop()

        generate(index + 1)

    generate(0)
    return result
```

At each index, the element is either included or excluded. Therefore, the number of possible subsets is

$$2^n.$$

Claim. *The algorithm generates every subset exactly once.*

Proof. Every subset corresponds to a sequence of n include-or-exclude decisions. The algorithm explores both decisions at every index, so every possible sequence is visited.

Different decision sequences select different sets of positions. Therefore, every subset is generated exactly once. □

The copy

```
current_subset[:]
```

is necessary because the same list is modified during later recursive calls.

7.5 Generate Parentheses and Pruning

For valid parentheses, the following conditions are maintained:

$$\text{closing count} \leq \text{opening count}$$

and

$$\text{opening count} \leq n.$$

If the closing count exceeds the opening count, that partial string can never become valid. Ending such branches early is called pruning.

8. Introduction to Dynamic Programming

Dynamic Programming has only been introduced at a basic level so far. The three problems I completed were:

- Climbing Stairs;
- Min Cost Climbing Stairs;
- House Robber.

These problems introduced the basic ideas of states, recurrence relations, base cases, overlapping subproblems, and tabulation.

8.1 Why Dynamic Programming Is Useful

Dynamic Programming is useful when a problem has:

1. **Overlapping subproblems:** the same smaller problem appears repeatedly.
2. **Optimal substructure:** the solution to a larger problem can be built from solutions to smaller problems.

A general DP process is:

1. Define what one state represents.
2. Write a recurrence connecting the state to earlier states.
3. Set the base cases.
4. Calculate states in a valid order.
5. Return the state representing the complete problem.

8.2 Memoisation and Tabulation

Memoisation starts with recursion and stores the result of every state. When a state is requested again, the saved result is reused.

Tabulation builds the answers iteratively, usually from smaller states to larger states.

The basic problems I solved used tabulation because the order of calculation was straightforward.

8.3 Climbing Stairs

At every move, one or two steps may be climbed.

Let

$$dp[i]$$

represent the number of ways to reach step i.

The final move to step i must come from either:

- step $i-1$ using one step;
- step $i-2$ using two steps.

Therefore,

$$dp[i] = dp[i-1] + dp[i-2].$$

```
def climb_stairs(n):
    if n == 1:
        return 1

```

```
    if n == 2:
        return 2

    dp = [0] * (n + 1)

    dp[1] = 1
    dp[2] = 2

    for i in range(3, n + 1):
        one_step_before = dp[i - 1]
        two_steps_before = dp[i - 2]

        dp[i] = (
            one_step_before
            + two_steps_before
        )

    return dp[n]
```

Claim. *For every i, $dp[i]$ equals the number of distinct ways to reach step i.*

Proof. Any valid path to step i ends with either a one-step move from $i-1$ or a two-step move from $i-2$. These two groups are disjoint because their final moves are different.

Therefore, the total number of paths is

$$dp[i-1] + dp[i-2].$$

The base cases $dp[1] = 1$ and $dp[2] = 2$ are correct, so the recurrence constructs the correct answer for all later steps. □

$$T(n) = O(n), \qquad S(n) = O(n).$$

8.4 Min Cost Climbing Stairs

Let

$$dp[i]$$

represent the minimum cost required to reach position i.

Position i can be reached from:

- position $i-1$, paying $\text{cost}[i-1]$;
- position $i-2$, paying $\text{cost}[i-2]$.

Therefore,

$$dp[i] = \min\left(dp[i-1] + \text{cost}[i-1], dp[i-2] + \text{cost}[i-2]\right).$$

```
def min_cost_climbing_stairs(cost):
    n = len(cost)

    dp = [0] * (n + 1)

    dp[0] = 0
    dp[1] = 0

    for i in range(2, n + 1):
        cost_from_one_step = (
```

```
            dp[i - 1] + cost[i - 1]
        )

        cost_from_two_steps = (
            dp[i - 2] + cost[i - 2]
        )

        dp[i] = min(
            cost_from_one_step,
            cost_from_two_steps
        )

    return dp[n]
```

Claim. *For each position i, $dp[i]$ stores the minimum possible cost required to reach that position.*

Proof. The final move to i must come from either $i-1$ or $i-2$. There are no other possible final moves.

By assuming the stored values for those earlier positions are minimal, the minimum of the two possible final transitions gives the minimum cost to reach i. □

8.5 House Robber

In this problem, adjacent houses cannot both be selected.

Let

$$dp[i]$$

represent the maximum amount that can be collected from houses 0 to i.

At house i, there are two choices:

1. skip house i, keeping $dp[i-1]$;
2. rob house i, adding its value to $dp[i-2]$.

Therefore,

$$dp[i] = \max\left(dp[i-1], dp[i-2] + \text{nums}[i]\right).$$

```
def rob(nums):
    n = len(nums)

    if n == 1:
        return nums[0]

    dp = [0] * n

    dp[0] = nums[0]
    dp[1] = max(nums[0], nums[1])

    for i in range(2, n):
        rob_current_house = (
            dp[i - 2] + nums[i]
        )

        skip_current_house = dp[i - 1]

        dp[i] = max(
            rob_current_house,
            skip_current_house
        )
```

```
23
24      return dp[n - 1]
```

Claim. *For every i, $dp[i]$ is the maximum amount obtainable from houses 0 through i without selecting adjacent houses.*

Proof. Any valid solution for houses up to i either includes house i or does not.

If house i is excluded, the best amount is $dp[i-1]$.

If house i is included, house $i-1$ cannot be selected, so the best amount is

$$dp[i-2] + \text{nums}[i].$$

Taking the maximum of these two complete possibilities gives the optimal answer. □

8.6 What I Learned from Basic DP

These problems showed me that the hardest part of DP is often defining the state clearly.

Once the meaning of $dp[i]$ is precise, the recurrence becomes easier to derive. I also learned that the base cases are part of the mathematical model, not only special programming conditions.

I have not yet covered advanced DP topics such as multidimensional states, subsequence DP, interval DP, or state compression. My current understanding is limited to basic one-dimensional recurrences.

9. Binary Trees and Binary Search Trees

Trees made recursion feel more natural because every subtree is another smaller tree.

9.1 Tree Traversals

The main depth-first traversal orders are:

$$\text{Preorder: Root, Left, Right,}$$

$$\text{Inorder: Left, Root, Right,}$$

$$\text{Postorder: Left, Right, Root.}$$

```python
def inorder_traversal(root):
    result = []

    def traverse(node):
        if node is None:
            return

        traverse(node.left)
        result.append(node.val)
        traverse(node.right)

    traverse(root)
    return result
```

Every node is visited once:

$$T(n) = O(n).$$

The recursion stack uses

$$O(h)$$

space, where h is the tree height.

9.2 Maximum Depth

```python
def max_depth(root):
    if root is None:
        return 0

    left_depth = max_depth(root.left)
    right_depth = max_depth(root.right)

    return 1 + max(left_depth, right_depth)
```

The recurrence is

$$\text{depth}(r) = 1 + \max\left(\text{depth}(r_{\text{left}}), \text{depth}(r_{\text{right}})\right).$$

Claim. *The function returns the length of the longest root-to-leaf path.*

Proof. Every root-to-leaf path continues through either the left subtree or the right subtree. The longest path through the left has length

$$1 + \text{depth}(r_{\text{left}}),$$

and the longest path through the right has length

$$1 + \text{depth}(r_{\text{right}}).$$

Taking the maximum gives the longest path. □

9.3 Level-Order Traversal

```python
from collections import deque

def level_order(root):
    if root is None:
        return []

    result = []
    nodes = deque([root])

    while nodes:
        level_size = len(nodes)
        current_level = []

        for i in range(level_size):
            node = nodes.popleft()
            current_level.append(node.val)

            if node.left is not None:
                nodes.append(node.left)

            if node.right is not None:
                nodes.append(node.right)

        result.append(current_level)

    return result
```

The queue ensures that all nodes on one level are processed before nodes on the next level.

9.4 Binary Search Trees

A Binary Search Tree satisfies

$$x < r \quad \text{for every } x \text{ in the left subtree,}$$

and

$$x > r \quad \text{for every } x \text{ in the right subtree.}$$

```python
def search_bst(root, target):
    current_node = root

    while current_node is not None:

        if current_node.val == target:
            return current_node

        if target < current_node.val:
            current_node = current_node.left
        else:
            current_node = current_node.right
```

```
13
14      return None
```

The running time is

$$O(h).$$

For a balanced BST,

$$h = O(\log n),$$

while for a skewed BST,

$$h = O(n).$$

9.5 Why Inorder Traversal Is Sorted

Claim. *The inorder traversal of a Binary Search Tree produces the values in increasing order.*

Proof. Every value in the left subtree is smaller than the root, while every value in the right subtree is larger.

The inorder traversal visits the sorted left subtree, then the root, then the sorted right subtree. Therefore, the complete result is sorted. □

10. Greedy Algorithms and Heaps

Greedy algorithms make the best immediate choice. Their implementations may be short, but the local decision must be proved safe.

10.1 Assigning Cookies

The strategy is:

1. sort the child requirements;
2. sort the cookie sizes;
3. give the smallest sufficient cookie to the least demanding child.

```python
def find_content_children(requirements, cookies):
    requirements.sort()
    cookies.sort()

    child_index = 0
    cookie_index = 0

    while (
        child_index < len(requirements)
        and cookie_index < len(cookies)
    ):
        if cookies[cookie_index] >= requirements[child_index]:
            child_index += 1

        cookie_index += 1

    return child_index
```

Using a larger cookie than necessary may reduce the options available for a more demanding child. The smallest sufficient cookie preserves larger cookies.

10.2 Interval Scheduling

Suppose the aim is to choose the maximum number of non-overlapping intervals. The greedy method selects the interval that finishes earliest.

Claim. *There exists an optimal solution whose first chosen interval is the earliest-finishing interval.*

Proof. Let G be the earliest-finishing interval. Consider an optimal solution whose first interval is O.

If $O = G$, the claim holds.

Otherwise, replace O with G. Since G finishes no later than O, every interval that was compatible after O is also compatible after G.

Thus, the replacement does not reduce the number of selected intervals. □

10.3 Jump Game

The algorithm maintains the farthest reachable index.

At index i,

$$\text{farthest} = \max\left(\text{farthest}, i + \text{nums}[i]\right).$$

If i is greater than the farthest reachable index, then index i cannot be reached.

10.4 Heaps

Python's `heapq` module implements a min-heap.

Typical costs are:

$$\text{minimum access} = O(1),$$

$$\text{insertion} = O(\log n),$$

$$\text{removal} = O(\log n).$$

10.5 Finding the k-th Largest Element

```python
import heapq

def find_kth_largest(nums, k):
    heap = []

    for number in nums:
        heapq.heappush(heap, number)

        if len(heap) > k:
            heapq.heappop(heap)

    return heap[0]
```

Invariant. *After processing any prefix, the heap contains the largest k values from that prefix, or all values if fewer than k values have been processed.*

At the end, the smallest value in this heap is the k-th largest overall.

$$T(n) = O(n \log k), \qquad S(n) = O(k).$$

11. Implementation Errors, Time Limits, and Memory

Some of the most useful lessons came from programs that failed during submission.

11.1 Index Errors

For a list of length n, the valid indices are

$$0, 1, \ldots, n-1.$$

The following loop is incorrect:

```
for i in range(len(nums) + 1):
    print(nums[i])
```

The final value of i is `len(nums)`, which is outside the valid range.

11.2 Negative Indices

Python allows negative indices.

```
last_value = nums[-1]
```

This is convenient, but it can hide pointer mistakes. If an index accidentally becomes -1, Python may access the final element instead of raising an immediate error.

11.3 Empty Containers

Calling `pop()` on an empty list or `popleft()` on an empty deque causes an error.

```
if stack:
    top_value = stack.pop()
```

11.4 None in Trees

Before accessing a node's value or children, the program must verify that the node is not `None`.

```
if node is None:
    return
```

11.5 Python List Slicing

A slice creates a new list.

```
part = nums[start:end]
```

If the slice contains m values, it requires

$$O(m)$$

time and memory.

Repeated slicing inside a loop can therefore make a solution much slower.

11.6 List Membership

Checking membership in a list may require scanning the complete list.

```
if number in nums:
    ...
```

The worst-case cost is

$$O(n).$$

If performed inside another loop, the total can become $O(n^2)$.

11.7 Copying and Shared Mutation

Passing a list to a function does not automatically create a copy.

```
def change_list(nums):
    nums.append(10)
```

This modifies the original list.

An independent copy can be created using

```
copied_nums = nums[:]
```

but copying a list of size n requires $O(n)$ time and memory.

11.8 Time Limit Exceeded

Common causes include:

- nested loops over large inputs;
- repeated linear searches;
- recalculating the same information;
- unnecessary sorting;
- repeated slicing;
- recursive exploration without pruning;
- using the wrong data structure.

A Time Limit Exceeded result often means that the overall approach must change.

11.9 Memory Limit Exceeded

Common causes include:

- very large lists or matrices;
- storing unnecessary intermediate results;
- repeated list copies;
- deep recursion;
- retaining data that is no longer needed.

12. Correctness Proofs and Invariants

A loop invariant is a statement that remains true during an algorithm.

A standard proof has three parts:

1. **Initialisation:** the invariant is true before the first iteration.
2. **Maintenance:** one iteration preserves the invariant.
3. **Termination:** when the loop ends, the invariant implies the required answer.

12.1 Insertion Sort Invariant

Before iteration i, the prefix from index 0 to $i-1$ is sorted.

Initially, the prefix contains one value and is therefore sorted. During each iteration, the current value is inserted into the correct position. When the algorithm ends, the sorted prefix is the complete list.

12.2 Binary Search Invariant

If the target exists, it remains inside the current search interval.

Initially, the interval is the full list. At each step, sorted order proves that one half cannot contain the target. When the interval becomes empty, the target does not exist.

12.3 Sliding-Window Invariant

After shrinking, the current window satisfies the required condition.

This invariant helps determine when the left pointer should move and when the answer may be updated.

12.4 Heap Invariant

For the k-th largest problem, the heap stores the largest k values seen so far.

The invariant directly explains why the heap root is the final answer.

12.5 DP State Invariant

For a tabulation-based DP algorithm, the usual invariant is:

> Before calculating state i, all previous states required by its recurrence have already been calculated correctly.

This explains why the order of filling the DP table matters.

12.6 Why Proofs Help with Debugging

Correctness proofs are also useful during debugging. When a program gives the wrong answer, I can ask:

- Did the binary-search interval still contain every possible answer?
- Was the sliding window valid when the answer was updated?
- Did the stack remain monotonic?
- Was the recursive state restored after backtracking?
- Did the heap still contain the correct candidates?
- Did the DP state represent what I intended?

This identifies which logical property failed rather than only checking random lines of code.

13. Current Progress and Final Reflection

I have become more comfortable with array traversal, frequency counting, prefix sums, standard binary search, basic recursion, elementary sorting, and simple stack problems.

I can also understand sliding windows, monotonic stacks, heaps, backtracking, binary search on the answer, and introductory Dynamic Programming when the main idea is explained. However, I still find it difficult to independently identify these patterns in unfamiliar medium-level questions.

13.1 Areas That Still Require Practice

The main areas I am continuing to work on are:

- recognising the correct sliding-window condition;
- defining recursive and DP states clearly;
- deciding what should be stored in a stack;
- proving that a greedy choice is safe;
- identifying monotonicity for binary search on the answer;
- avoiding unnecessary copying and repeated work;
- combining multiple techniques in one question.

13.2 How I Plan to Improve

A useful practice method for me is:

1. attempt the question independently for a fixed amount of time;
2. write a brute-force solution even if it is inefficient;
3. identify the repeated work;
4. request only a hint or intuition when necessary;
5. implement the improved approach independently;
6. trace the algorithm manually;
7. write the complexity and invariant;
8. solve a similar question without assistance.

This helps turn external guidance into actual learning.

13.3 Final Conclusion

The main lesson from the course so far is that solving a programming problem is not only about obtaining the correct output.

A complete solution should answer:

- Why does the method work?
- What information is maintained?
- What repeated work has been removed?
- How does the running time grow?
- How much memory is used?
- Which edge cases may cause failure?

Some of the most interesting algorithms were those where a small mathematical observation produced a large improvement. Prefix-sum cancellation, equal remainders, sliding-window amortisation, binary-search monotonicity, monotonic-stack behaviour, recursion induction, greedy exchange arguments, and DP recurrences all showed that mathematical reasoning directly supports efficient code.

Using ChatGPT for initial intuition has helped me approach questions that would otherwise feel inaccessible. However, I have also learned that reading a solution is much easier than producing one.

I therefore try to use explanations as a starting point and then rewrite, trace, test, and justify the algorithm myself.

I am still developing confidence with medium-level problems and have only started basic Dynamic Programming. However, my approach has become more structured. I now compare multiple solutions, pay attention to constraints, analyse time and memory, define states and invariants, and think about edge cases before submitting code. This change in problem-solving method has been the most important outcome of the course so far.